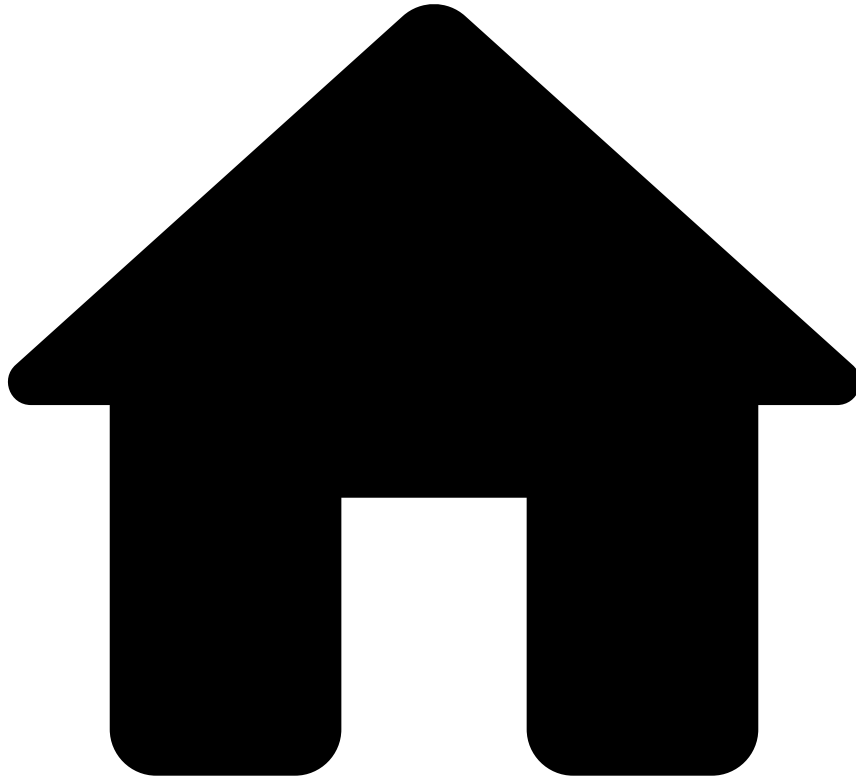


-



- [Deploying a project into Production](#)
- [Designing a workflow](#)
- Adding custom services

On this page

Adding custom services

Was this page helpful? 

Why

Custom Services implement custom logic which you can add to your runtime [Workflows](#). Other than Input Adapters you can use other **Custom Services** that react to messages as they are processed in the **Workflow**. For example, **Action Listeners**, merging decisions from different [Models](#), implementing caching, processing events to be displayed in dashboards, etc.

Depending on what the **Custom Service** is designed for, it may be connected in the data flow together with out-of-the-box services such as **Plan**, **Scoring**, and **Rules**, or just as a disconnected element that adds extra logic without affecting the flow of events.

How To

To add and configure a **Custom Workflow Element** to your **Workflow**, for example, to process events immediately before reaching the end of the **Workflow**:

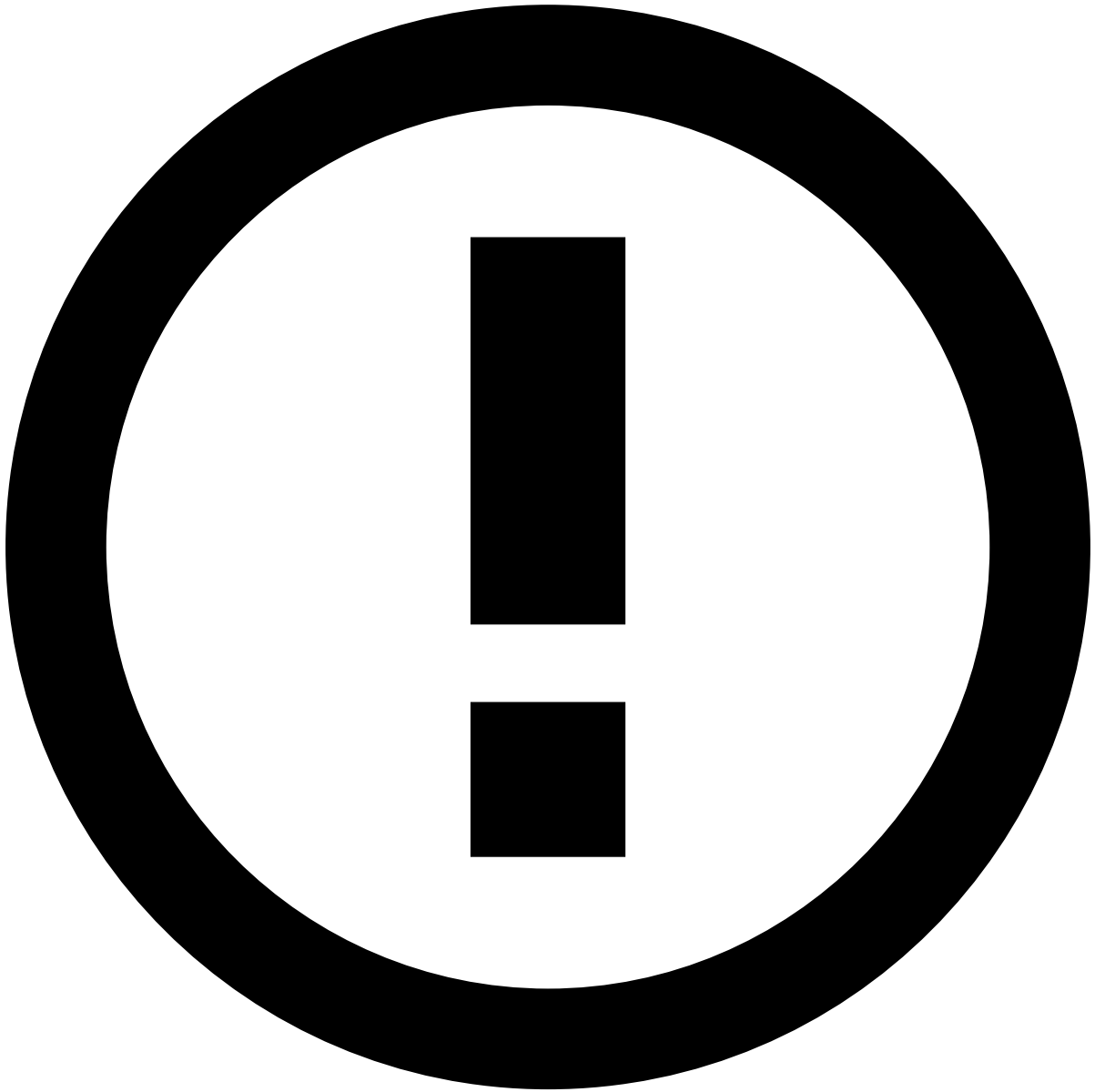
1. In the **Workflows** tab, select an existing **Workflow**.
2. Drag a new **Custom Workflow Element** to the **Diagram** canvas.
3.  Edit the **Custom Workflow Element**.
4. In the **Main** tab:
 1. Select the **Custom Service** you want to apply in your **Workflow**.
 2. If the **Custom Service** has been developed to contribute to a [Workflow Outcome](#), select an **Outcome**.
After being processed at a **Custom Workflow Element**, the `<outcome_name>` and `_<outcome_name>score` fields in the event schema will reflect the processing.
5. In the **Schemas** tab, if the **Custom Service** operates over the data itself:
 1. Confirm that the **Service Input** and **Service Output** are consistent with the data you expect for the **Custom Service**.
 2. Make sure the **Service Input** and **Service Output** are correctly mapped from the **Event Schema**, for each real-time event moving along the **Workflow**.
6. In the **Properties** tab, set the values for each property to define the service configuration in your **Workflow**.
Properties and default values depend on how the **Custom Service** is developed.
7.  **Connect** the previous service in the **Workflow** to the **Custom Workflow Element**, if it is part of the data flow.
For example, if you implement an **Action Listener** in a **Custom Service**, you just need to add it to the **Diagram** canvas, without having to **Connect** it.

Solving schema conflicts

If Pulse invalidates your [Workflow](#) due to schema mapping conflicts in a Workflow Service, you must solve them as follows:

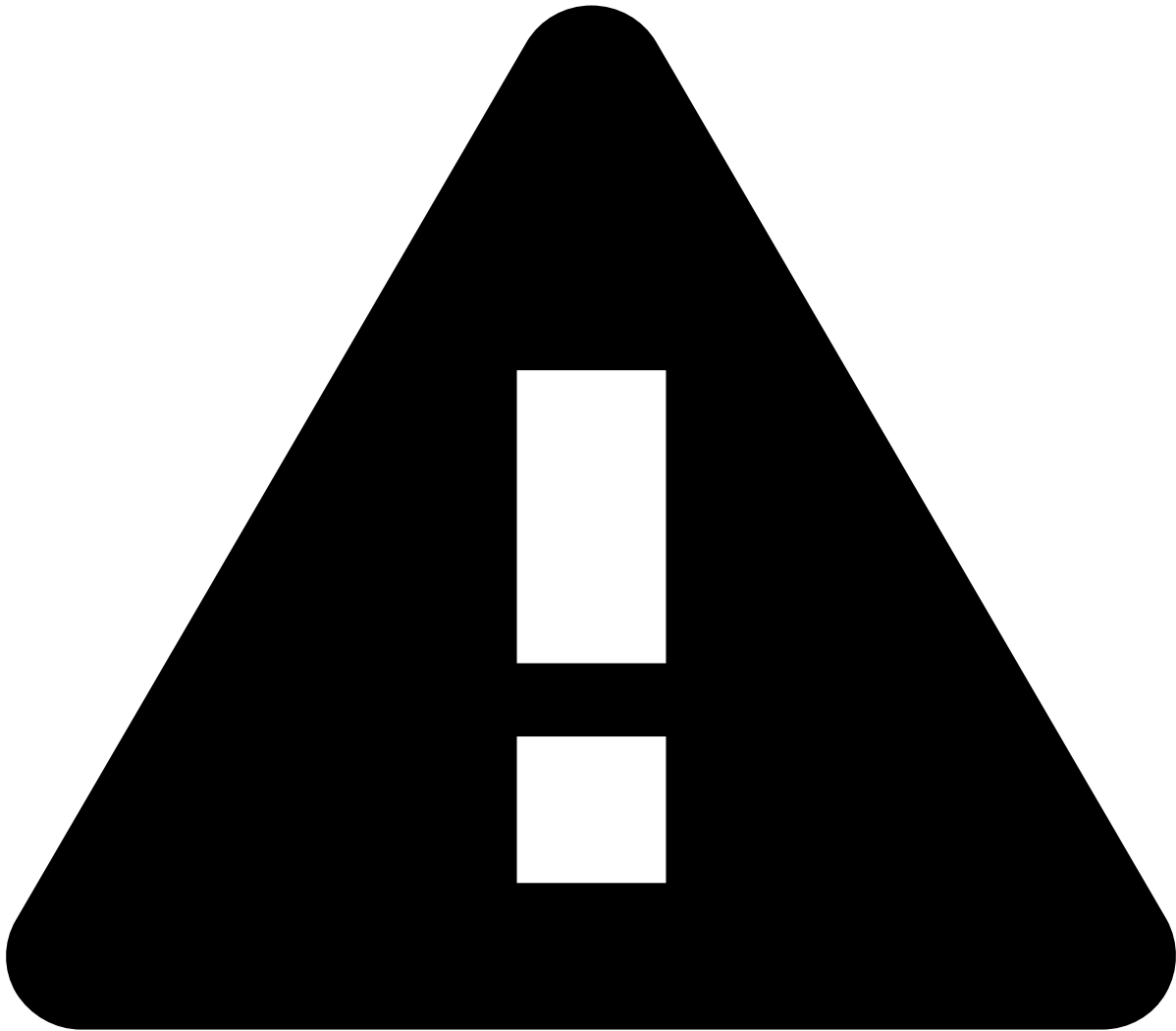
1.  Edit the **Service**.
2. Select the **Schemas** tab.
3. Look for a count of **Input Conflicts** and **Output Conflicts** above 0.
4. Select each **Conflict** filter to see which fields could not be mapped. Alternatively, select the **Service** filter and locate the conflict fields in the **Event Schema**. These are marked as *Missing Input* or *Missing Output*.
5. If the **Event Schema** contains a field that you can map to the **Service Input** or **Output**, manually enter the name of that field.

The following images display schema conflicts during resolution:



Conflicts count

Note that Pulse will still display the original conflicts count even after you have solved them.



Removed fields

If a **Plan** you configured for a **Plan Service** removes fields from the workflow schema, then you must ensure that subsequent services in the **Workflow** do not require those fields. Otherwise, Pulse will generate schema conflicts, invalidate your **Workflow**, and you will have no available fields to map from.

In these cases, you should redesign your **Plan**, articulate it with the remaining Data Science work to be used in runtime, and redeploy it to your **Workflow**.

Next Steps📖

Proceed with the designing of your Workflow by adding, configuring and connecting other services. Check the following pages:

- [Adding input adapters](#)
- [Adding plans](#)
- [Adding models](#)
- [Adding rules](#)